

Tema 6.1:
**Documentación de aplicaciones Android
Dokka**



Documentación de Proyectos

La documentación se puede hacer de muchas formas, desde comentarios simples que acompañan al código, hasta sitios web complejos como la documentación para desarrolladores de Android (<https://developer.android.com/develop>). El objetivo es cubrir aspectos como la API de la plataforma, las herramientas de compilación, muestras, etc., todo es documentación.

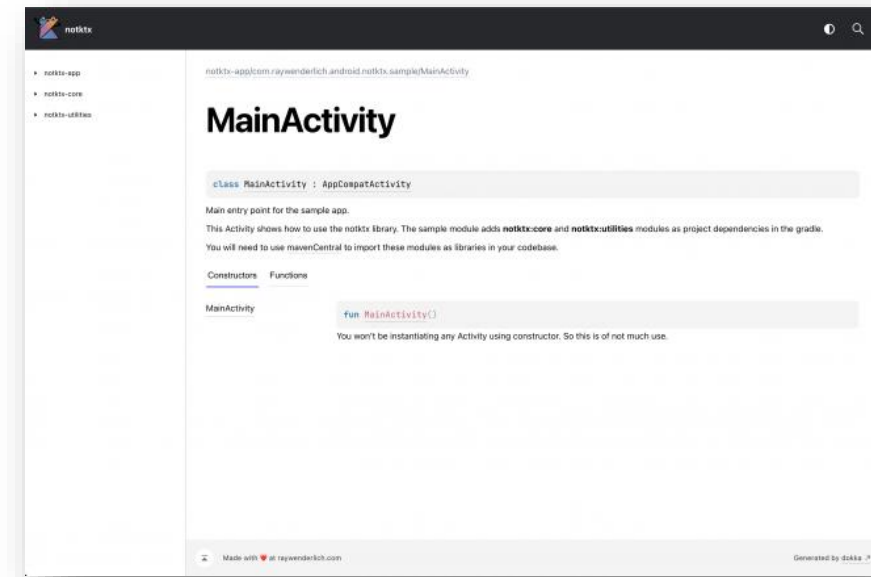
La documentación del código es la descripción que ayuda a comprender qué hace cada código en la aplicación. Por ejemplo, ayuda a proporcionar una visión general clara de lo que hace cada variable, qué función específica en el proyecto es, si tiene un parámetro o un tipo de retorno, y qué efectos tiene en el código cuando los detalles están bien documentados.

Normalmente la usaremos para **comprender qué variables, clases, objetos, funciones, etc.** se utilizan en la aplicación y cuál es su propósito, y proporcionar información sobre si tiene parámetros, destinatarios o enlaces externos. Una vez que obtiene suficiente información sobre el código, se vuelve fácil trabajar en el código, lo que reduce el tiempo que lleva resolver e implementar una solución en la aplicación.

Documentación de Proyectos

El principal público objetivo de este tipo de documentación suelen ser **otros desarrolladores**, por lo que documentar un proyecto puede ser una excelente manera de comprender cómo documentar las bases de código. Facilitar el uso de su proyecto a otros desarrolladores es una tarea desafiante pero gratificante, y una buena documentación juega un papel importante en ello.

La documentación generada en HTML se verá algo así:



Documentación de Proyectos

Para ayudar en estas tareas de documentación es importante que, **durante el desarrollo del proyecto**, ir eligiendo nombres descriptivos para variables y funciones, es el primer paso hacia un código base bien documentado.

Y también puedes ir aún más lejos y **agregar comentarios** que describan la función.

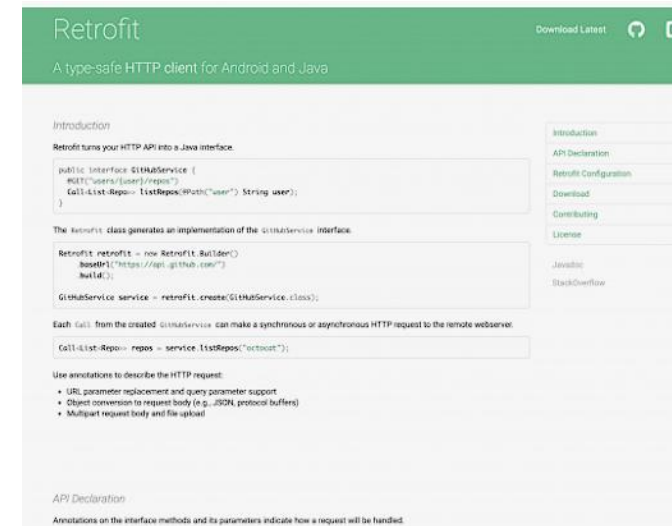
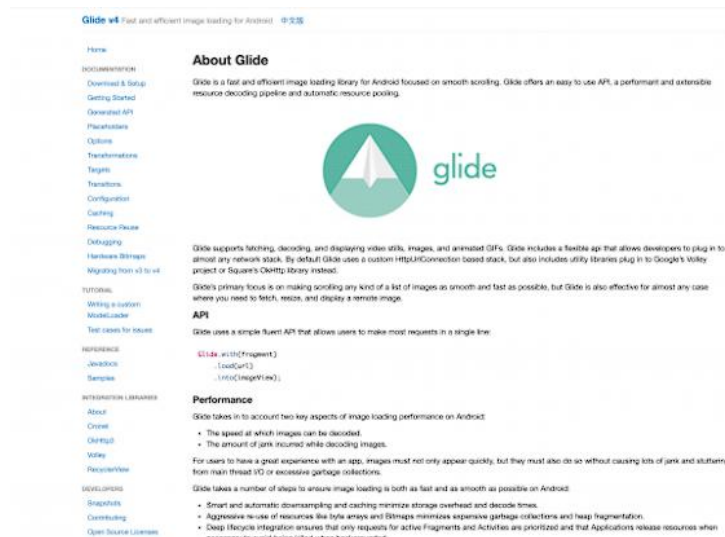
Cuando documentas tu código, ayudas a nuevos usuarios y contribuyentes a confiar y comprender mejor tu proyecto. En un entorno profesional, una buena documentación ayuda a los nuevos desarrolladores de su equipo, así como a otros equipos, a comenzar con su proyecto rápidamente.

La documentación también te ayuda. Revisar su antigua base de código línea por línea después de un tiempo puede llevar mucho tiempo. Entonces, al documentarlo, también estás ayudando a tu yo futuro.

Documentación de Proyectos

Aprender de los ejemplos

En el mundo de Android, Glide y Retrofit son bastante famosos por cargar imágenes y realizar llamadas de red, y sus páginas de documentación son realmente buenas.



¿Qué es Dokka?

En **Java**, **Javadoc** se usa para documentar código Java. Antes de **Dokka**, no había una herramienta adecuada para documentar Kotlin. Dokka es una herramienta de documentación similar a Javadoc. La principal diferencia entre Javadoc y Dokka es que Dokka es compatible con la documentación de **Kotlin y Java**. La documentación de Dokka es posible con KDoc.

KDoc es el Idioma se utiliza para documentar el código Kotlin. Los comentarios adecuados son necesarios para crear una buena documentación. KDoc también ayuda a comentar usando etiquetas de bloque y rebajas. Básicamente es un complemento utilizado en Android Studio que ayuda con la documentación. Podemos usar KDoc de la siguiente manera:

- Los comentarios de KDoc comenzaban con `/**` como etiqueta de apertura y terminaban con la etiqueta de cierre `*/`
- KDoc admite **etiquetas** de bloque, así como Markdown en línea y HTML para proporcionar información adicional sobre los elementos incrustados con el símbolo `@` (se verán posteriormente)

¿Qué es Dokka?

Por tanto, **Dokka** es una herramienta para generar documentación basada en comentarios **Javadoc** o **Kdoc** (Kotlin Docs) que producirá el documento generado en formatos **HTML**, **GFM** (GitHub Flavored Markdown) o formato **Javadoc**.

Por ejemplo, podría ser utilizado para generar documentación para cualquier aplicación o biblioteca de Android.

Para integrar Dokka en este proyecto, usarás **gradle** .

[Github oficial de Kotlin Dokka](#)

¿Cómo lo instalamos?

1. En tu archivo **build.gradle** a nivel de **proyecto**, agrega el plugin de Dokka y **sincroniza**:

```
plugins {  
    .  
    .  
    id("org.jetbrains.dokka") version "1.9.10"  
}  
subprojects {  
    apply(plugin = "org.jetbrains.dokka")  
}
```

2. Agrega también el classpath de Dokka a tu archivo **build.gradle** a nivel de **proyecto y sincroniza**:

```
dependencies {  
  
    dokkaPlugin("org.jetbrains.dokka:android-documentation-plugin:1.9.10")  
}
```

¿Ejecutar Dokka?

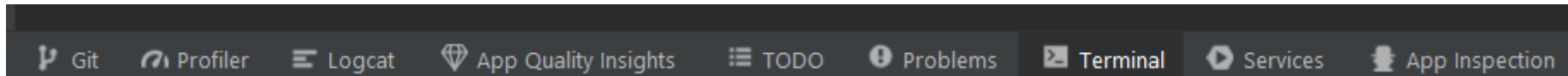
Para ejecutar Dokka y generar la documentación en HTML, GFM (Markdown con sabor a GitHub) y Javadoc tenemos que **ejecutar** diversos comandos en el **terminal** de Android Studio:

Para Windows: utiliza **gradlew.bat** y para Linux o Mac: utiliza **./gradlew**

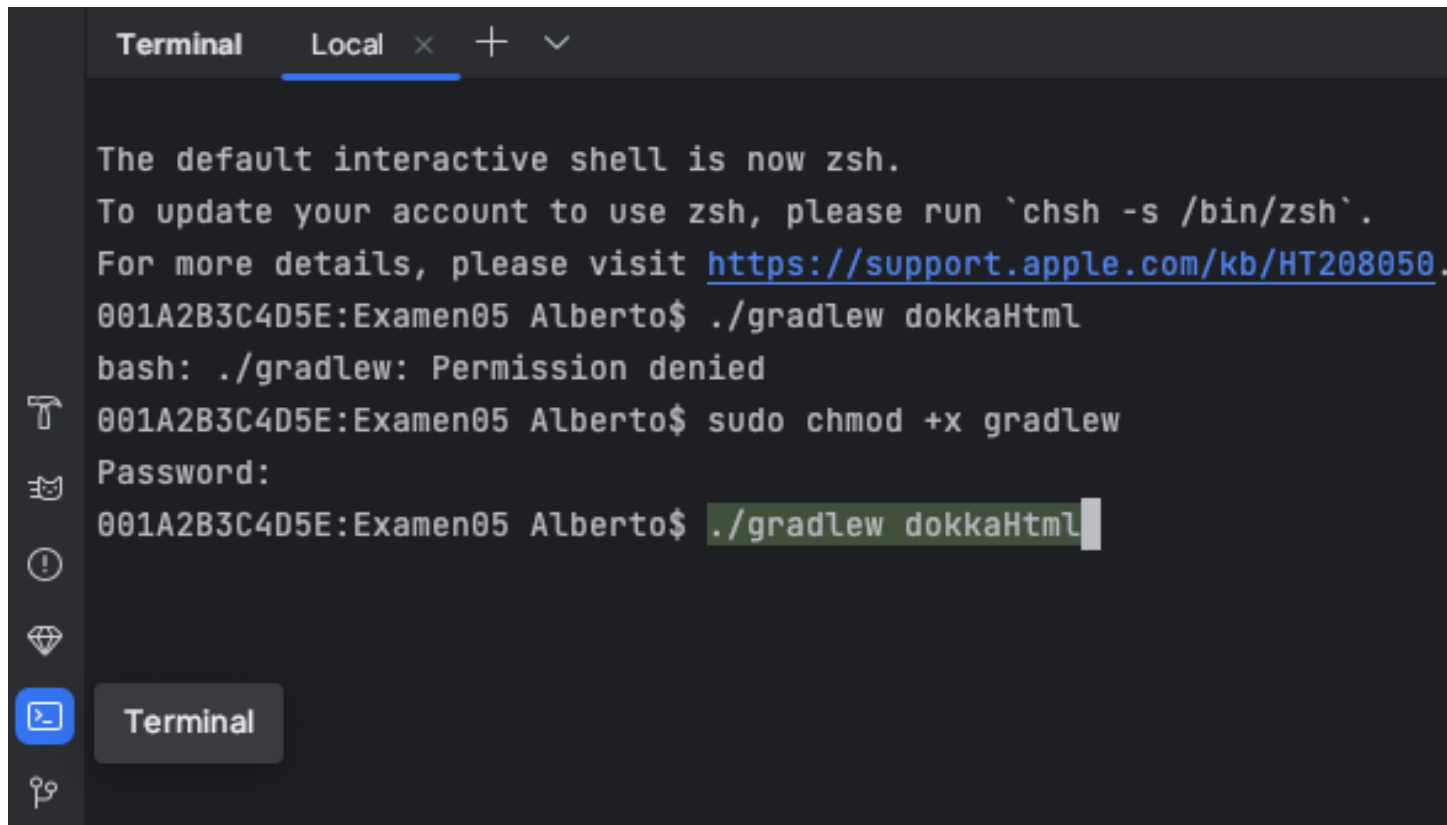
Y según lo que queramos:

- Ejecuta: **./gradlew.bat dokkaHtml** para generar Documentos HTML.
- Ejecuta: **./gradlew.bat dokkaGfm** para generar Documentos GFM.
- Ejecuta: **./gradlew.bat dokkaJavadoc** para generar Javadoc.

La documentación generada estará en el directorio **app\build\dokka**.



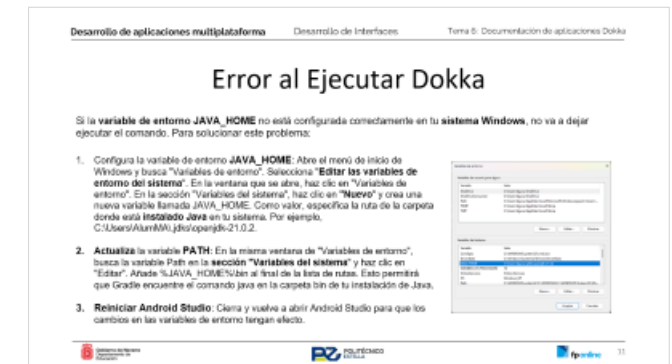
¿Ejecutar Dokka?



```
Terminal Local x + v

The default interactive shell is now zsh.
To update your account to use zsh, please run `chsh -s /bin/zsh`.
For more details, please visit https://support.apple.com/kb/HT208050.
001A2B3C4D5E:Examen05 Alberto$ ./gradlew dokkaHtml
bash: ./gradlew: Permission denied
001A2B3C4D5E:Examen05 Alberto$ sudo chmod +x gradlew
Password:
001A2B3C4D5E:Examen05 Alberto$ ./gradlew dokkaHtml
```

Si te da error al ejecutarlo:



Desarrollo de aplicaciones multiplataforma Desarrollo de Interfaces Tema 6: Documentación de aplicaciones Dokka

Error al Ejecutar Dokka

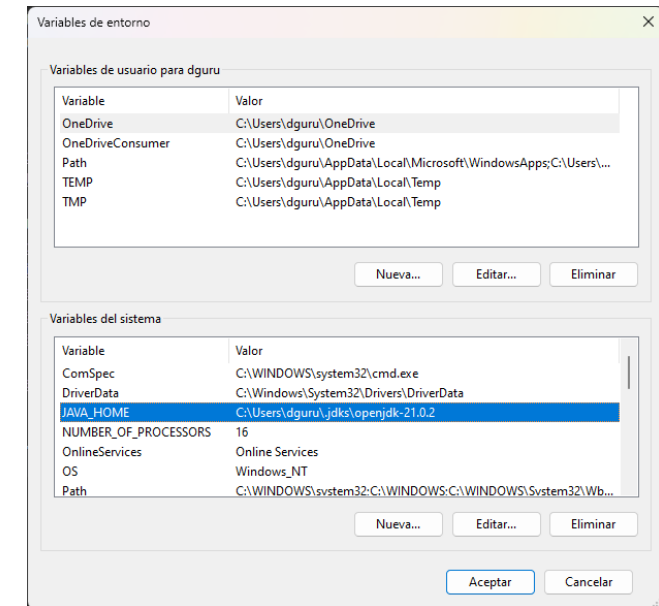
Si la **variable de entorno JAVA_HOME** no está configurada correctamente en tu **sistema Windows**, no va a dejar ejecutar el comando. Para solucionar este problema:

1. Configura la variable de entorno **JAVA_HOME**: Abre el menú de inicio de Windows y busca "Variables de entorno". Selecciona "Editar las variables de entorno del sistema". En la ventana que se abre, haz clic en "Variables de entorno". En la sección "Variables del sistema", haz clic en "Nuevo" y crea una nueva variable llamada **JAVA_HOME**. Como valor, especifica la ruta de la carpeta donde está **instalado Java** en tu sistema. Por ejemplo, `C:\Users\AlumK00\jdk\openjdk-21.0.2`.
2. Actualiza la variable **PATH**: En la misma ventana de "Variables de entorno", busca la variable **Path** en la sección "Variables del sistema" y haz clic en "Editar". Añade `%JAVA_HOME%\bin` al final de la lista de rutas. Esto permitirá que Gradle encuentre el comando **java** en la carpeta bin de tu instalación de Java.
3. Reiniciar Android Studio: Cierra y vuelve a abrir Android Studio para que los cambios en las variables de entorno tengan efecto.

Error al Ejecutar Dokka

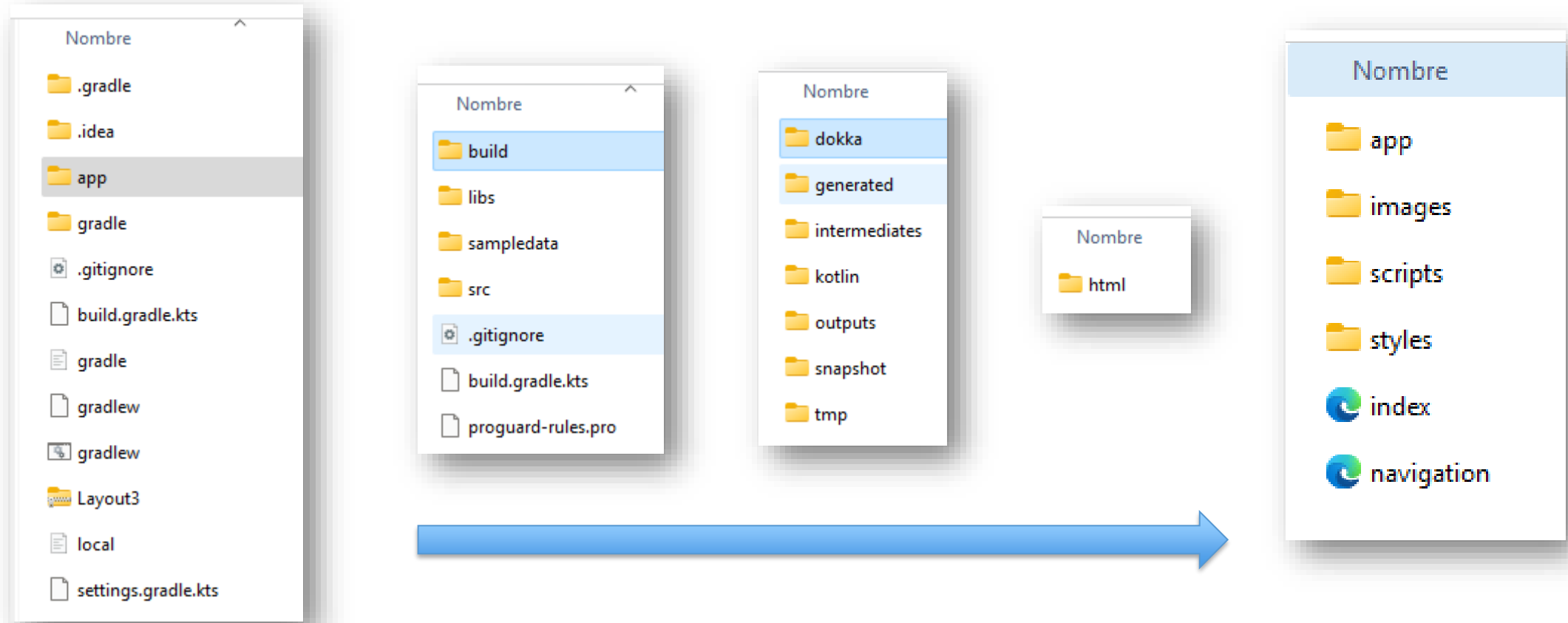
Si la **variable de entorno JAVA_HOME** no está configurada correctamente en tu **sistema Windows**, no va a dejar ejecutar el comando. Para solucionar este problema:

1. Configura la variable de entorno **JAVA_HOME**: Abre el menú de inicio de Windows y busca "Variables de entorno". Selecciona "**Editar las variables de entorno del sistema**". En la ventana que se abre, haz clic en "Variables de entorno". En la sección "Variables del sistema", haz clic en "**Nuevo**" y crea una nueva variable llamada **JAVA_HOME**. Como valor, especifica la ruta de la carpeta donde está **instalado Java** en tu sistema. Por ejemplo, `C:\Users\AlumMA\.jdk\openjdk-21.0.2`.
2. **Actualiza** la variable **PATH**: En la misma ventana de "Variables de entorno", busca la variable Path en la **sección "Variables del sistema"** y haz clic en "Editar". Añade `%JAVA_HOME%\bin` al final de la lista de rutas. Esto permitirá que Gradle encuentre el comando java en la carpeta bin de tu instalación de Java.
3. **Reiniciar Android Studio**: Cierra y vuelve a abrir Android Studio para que los cambios en las variables de entorno tengan efecto.

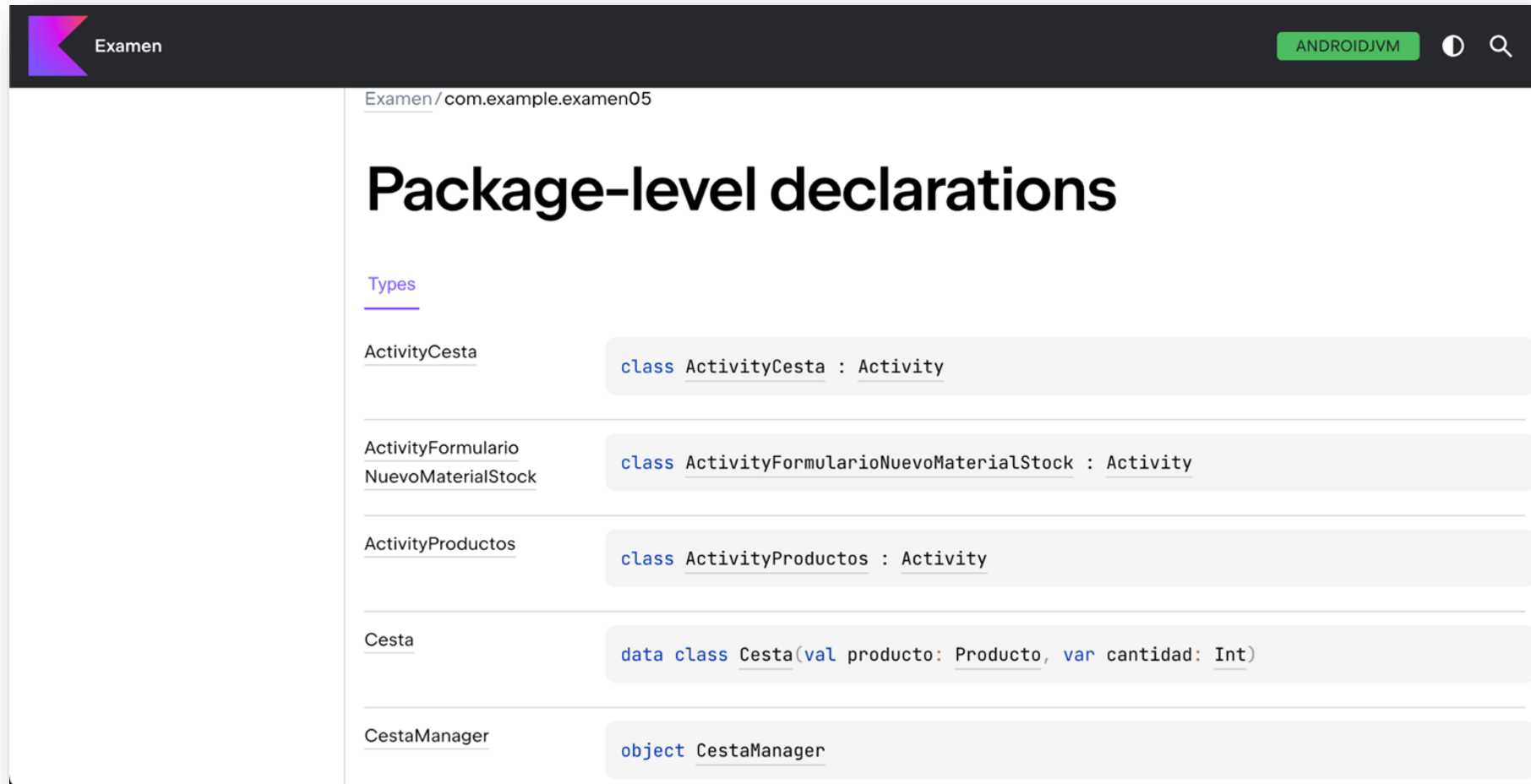


¿Qué se genera?

La **documentación html** generada está en la carpeta “build\dokka” de tu proyecto:



¿Qué se genera?

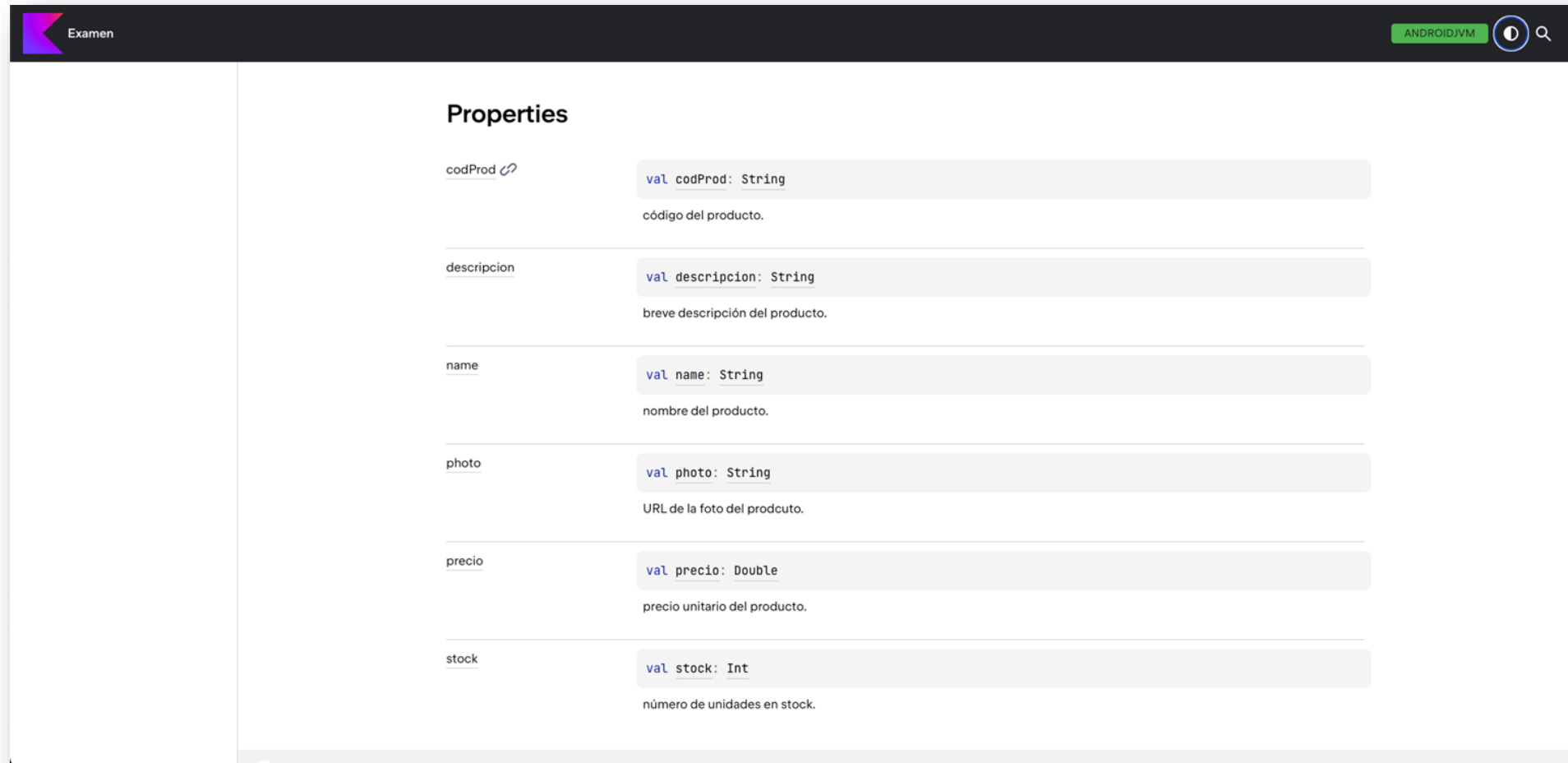


Comentarios KDoc o Javadoc

Al igual que con Javadoc, **los comentarios KDoc** tienen que comenzar con `/**` y terminan con `*/`. Cada línea del comentario puede comenzar con un asterisco, que no se considera parte del contenido del comentario.

```
/**
 * Definición de un producto.
 *
 * Esta clase sirve para definir los atributos que tiene un Producto.
 *
 * @param name the type of a member in this group.
 * @property name nombre del producto.
 * @property precio precio unitario del producto.
 * @property photo URL de la foto del producto.
 * @property codProd código del producto.
 * @property stock número de unidades en stock.
 * @property descripcion breve descripción del producto.
 */
data class Producto(
    val name: String,
    val precio: Double,
    val photo: String,
    val codProd: String,
    val stock: Int,
    val descripcion: String
)
```

Comentarios KDoc o Javadoc



The screenshot shows a web interface with a dark header. On the left, there's a purple 'K' logo and the word 'Examen'. On the right, there's a green 'ANDROIDJVM' button and a search icon. The main content area is titled 'Properties' and lists six properties:

Property Name	Kotlin Code	Description
<code>codProd</code>	<code>val codProd: String</code>	código del producto.
<code>descripcion</code>	<code>val descripcion: String</code>	breve descripción del producto.
<code>name</code>	<code>val name: String</code>	nombre del producto.
<code>photo</code>	<code>val photo: String</code>	URL de la foto del producto.
<code>precio</code>	<code>val precio: Double</code>	precio unitario del producto.
<code>stock</code>	<code>val stock: Int</code>	número de unidades en stock.

Etiquetas

@param name

Documenta un parámetro de valor de una función o un parámetro de tipo de una clase, propiedad o función. Para separar mejor el nombre del parámetro de la descripción, si prefieres, puedes encerrar el nombre del parámetro entre corchetes. Por lo tanto, las siguientes dos sintaxis son equivalentes:

```
@param name description. @param[name] description.
```

@return

Documenta el valor de retorno de una función.

@constructor

Documenta el constructor principal de una clase.

@receiver

Documenta el receptor de una función de extensión.

Etiquetas

@property name

Documenta la propiedad de una clase que tiene el nombre especificado. Esta etiqueta se puede usar para documentar propiedades declaradas en el constructor principal, donde colocar un comentario de documentación directamente antes de la definición de la propiedad sería incómodo.

@throws class, @exception class

Documenta una excepción que puede ser lanzada por un método. Dado que Kotlin no tiene excepciones verificadas, tampoco se espera que todas las excepciones posibles estén documentadas, pero aún así puedes usar esta etiqueta cuando proporcione información útil para los usuarios de la clase.

@sample identifier

Incrusta el cuerpo de la función con el nombre cualificado especificado en la documentación para el elemento actual, para mostrar un ejemplo de cómo se podría usar el elemento.

Etiquetas

@see identifier

Agrega un enlace a la clase o método especificado al bloque de Ver también de la documentación.

@author

Especifica el autor del elemento que se está documentando.

@since

Especifica la versión del software en la que se introdujo el elemento que se está documentando.

@suppress

Excluye el elemento de la documentación generada. Se puede usar para elementos que no forman parte de la API oficial de un módulo pero aún deben ser visibles externamente.

Cambiar el directorio

En tu archivo **build.gradle** a nivel de la aplicación, agrega:

```
    .  
    .  
    .  
    kotlinOptions {  
        jvmTarget = "1.8"  
    }  
  
    tasks.dokkaHtml.configure {  
        outputDirectory.set(file("../documentation/html"))  
    }  
  
    dependencies {  
  
        implementation("androidx.core:core-ktx:1.12.0")  
        implementation("androidx.appcompat:appcompat:1.6.1")  
        .  
        .  
        .  
    }
```

Javadoc

El **formato de salida Javadoc** de Dokka es similar al formato HTML Javadoc de Java .

Intenta imitar visualmente las páginas HTML generadas por la herramienta Javadoc, pero no es una implementación directa ni una copia exacta.

Todo el código y las firmas de Kotlin se representan desde la perspectiva de Java. Esto se logra con nuestro complemento Kotlin como Java Dokka , que viene incluido y se aplica de forma predeterminada para este formato.

Aparecerá en una carpeta similar a la de HTML.

The screenshot shows a Dokka-generated Javadoc page for the class `ArrayListSequenceReader`. The page has a navigation bar at the top with links: OVERVIEW, PACKAGE, CLASS (highlighted), TREE, DEPRECATED, INDEX, and HELP. Below the navigation bar is a section titled "ALL CLASSES". The main content area shows the package name, the class name `ArrayListSequenceReader`, and the interfaces it implements: `java.lang.Iterable`, `org.biojava.nbio.core.sequence.template.Accessioned`, and `org.biojava.nbio.core.sequence.template.Sequence`. The class signature is shown as `public class ArrayListSequenceReader<C> extends Compound<> implements SequenceReader<C>`. A brief description follows: "Stores a Sequence as a collection of compounds in an ArrayList". Below this is a "Field Summary" section with a "Fields" tab. The field list shows a single field: `public CompoundSet<C>`. Finally, there is a "Constructor Summary" section with a "Constructors" tab. The constructor list shows three constructors: `ArrayListSequenceReader()`, `ArrayListSequenceReader(List<C> compounds, CompoundSet<C> compoundSet)`, and `ArrayListSequenceReader(String sequence, CompoundSet<C> compoundSet)`.